

Creating an Asteroid Shape Model Educational Program & User-Friendly Interface

Evan Blosser, OU ID: 113489794

July 19, 2026

Abstract

This work aimed to create an asteroid gravitational field simulation and incorporate this into a Graphic User Interface (GUI) to create a user-friendly educational tool. This was accomplished by using multiple programs in both Python and C programming languages as well as shape models obtained from the 3d-asteroids.space catalog of all known asteroids. The end result is a program that incorporates 26 named asteroids and displays their shape along with the center of mass (CM) of the tetrahedrons that make up the shape model of the asteroid.

1 Introduction

A vast majority of asteroids are considered to have irregularly shaped bodies that require more complex methods for evaluating their gravitational fields. Most of their shape models are made using data from Radar Imaging and Light-curve Inversion [1]. Yet, these are not extremely accurate as compared to the models of previously visited asteroids that used Spacecraft imaging to define the exact shape. Because of this, most of the asteroids within the catalog are rough estimations of their shapes. This brings about the need for more research into analyzing the gravitational fields with the information that is avail-

able currently.

Every shape model for any of the asteroids within the catalog is either in Standard Triangle Format (PLY) or Alias Waveform Format (OBJ). For this project, the latter was found to be the ideal file to use as the PLY format requires separate graphics tools. This is because both formats can define additional texture data, yet the PLY format is far more detailed with a more complex data file as compared to the OBJ format. The PLY format was found to be incomprehensible in a basic text file format, yet a link to MeshLab was located near all download locations within the catalog and capable of converting these into OBJ files[1]. This would come in use later during the processing of the 26 asteroids as some needed to be converted from PLY to OBJ rather than using the direct OBJ download file from the catalog.

The OBJ format defines the vertices for each tetrahedron along with their respective faces as either 'v' or 'f', where each 'v' corresponds to an x-y-z vertex coordinate and each 'f' defined the 3 vertices that it was assigned to. These vertices and faces each combine to form individual tetrahedrons that make up the shape model's polyhedron. The texture data within the OBJ format are designated with 'vt' or 'vn' which could be ignored when processing the file by scanning only for 'v' if present in the file to extract the basic vertices locations and their respective faces[2]. For

these reasons, the project utilized the OBJ format for processing the asteroid shape models for analysis.

There are several methods for analyzing the gravitational potentials of irregularly shaped bodies. The focus of this work will be on the Mass Concentration, or MASCON, method of analysis. MASCON itself can be denoted as MASCON I, III, or VIII which signifies how each tetrahedron can be divided into subsequent portions. MASCON I is the most simplistic as each tetrahedron remains as a single shape from the outside face to the CM of the polyhedron. MASCON III and VIII can be utilized to set layers of the shape model, yet these are more complex and will not be covered within this project. However, the MASCON I model can be developed into these more complex models in order to approximate different densities and compositions of asteroids [3]. For this reason, the MASCON method was utilized within this project so that it could be developed further in future projects. Furthermore, this can be expanded upon using the “two-layer lumped-core approach” in order to create a two-layered asteroid shape model in MASCON [4].

2 Finding Properties of Asteroids

In order to analyze each individual Asteroid shape model, Brian Mirtich’s *volInt.c* program which was created on January 26th, 1996 [5]. In his work, Mirtich showed that under the assumptions of constant density the mass integrals could be rewritten as ten volumetric integrals. These volumetric integrals were then used to calculate the center-of-mass, volume, and inertia tensor of any polyhedron. However, in order for these volumetric integrals to be calculated by the C

language Mirtich had to utilize several theorems to reduce the integrals into summations.

First, the divergence theorem was used to reduce the three-dimensional integral over volume into an integral over the area of each planar face. However, these integrals still remain within the three-dimensional space. Thus the second unnamed theorem used was presented along with the proof showing how these integrals over the faces can be projected onto two-dimensional planes. This then allowed Mirtich to use Green’s theorem in order to further simplify the integrals over the projected faces into a summation of line integrals. The final unnamed theorem, also presented along with the proof, shows how the line integrals could be directly evaluated through summation by the use of the vertex coordinates.

Mirtich’s work allowed the analysis of polyhedron shape models by reducing volumetric integrals to be simplified to summations which could, in turn, be coded into the C language. The program *volInt.c*, which was presented along with his paper, is located within the OpenFOAM-2.1.x GitHub repository and used for the main calculations for the Center of Mass (CM) for each asteroid’s polyhedron shape model [6]. The program was also used to produce various other properties for the asteroid such as the inertia tensor that could be presented to the user within the GUI and could be used for further calculations of orientation and orbital analysis.

2.1 Creating *volInt.c* input file from OBJ format

The first main hurdle in using the *volInt.c* was constructing a program that could convert OBJ files into the specified data file structure defined by Mirtich within the README file that accompanies the program. The file structure defined is similar to that of

the OBJ format, yet there were still modifications such as removing the 'v' and 'f' designations. Mirtich's *volInt.c* required the total number of vertices, followed by the list of each x-y-z vertex coordinates. After this the total number of faces needed to be listed before listing each face's corresponding vertex. In addition, before each row of face's there needed to be a total number of vertices assigned to each face which in the case of tetrahedrons was 3 vertices per face [6].

The following is a sample of the OBJ file for Apophis (99942) which was the chosen asteroid used to test the program:

Apophis (99942) OBJ file in text format:

```
# Object 99942_Apophis.obj
#
# Vertices: 1014
# Faces: 2024
#
####
v -0.058457 0.358872 0.591007
v -0.066710 0.355750 0.591644
-----etc-----
v 0.179480 0.104379 -0.544479
v 0.126730 0.058077 -0.546444
# 1014 vertices, 0 vertices normals

f 1 2 3
f 1 3 4
```

In order to convert this to the desired input file for *volInt.c* the program *obj2MirtichData.py* was created, see Appendix A for the Python program location. This program scans each row for either 'v' or 'f' and disregards any additional information. This was carried out by first counting the numbers of 'v' and 'f' and using these counts to sort the data taken from the OBJ file as vertex data or face data. The counts were then placed in their designated spots in the data file, either before the vertex list or

the face list. Additionally, as each model utilizes a tetrahedron, with 3 vertices assigned to each face, a column filled with 3s was added to the face data as specified by the README for *volInt.c*. The following is a sample of the generated data file created by *obj2MirtichData.py* and was used as the input file for *volInt.c*:

Apophis (99942) in Mirtich defined data file format:

```
1014
-0.05845700 0.35887200 0.59100700
-0.06671000 0.35575000 0.59164400
-----etc-----
0.17948000 0.10437900 -0.54447900
0.12673000 0.05807700 -0.54644400
2024
3 1 2 3
3 1 3 4
```

Once the data file was structured correctly it could be used as an input file within *volInt.c* to calculate the necessary CM of the polyhedron along with other properties of the asteroid. It should be noted that any comments using the '#' symbol or blank spaces were disregarded from the data within Python. Thus, the program *obj2MirtichData.py* could be reused for all OBJ files located in the 3d-asteroids.space catalog. A caveat to this is that some OBJ files downloaded from the catalog are in an incorrect format which produced an error while running *obj2MirtichData.py*. However, this is easily remedied by downloading the PLY instead and converting it to OBJ using the *MeshLab* linked on the website. This ensures that the OBJ is structured properly as 99942_Apophis.obj as shown above.

2.2 Modifying *volInt.c*

The main assumption of this project is that all asteroid densities were assumed to be 1.0

with assumed units. The *volInt.c* code was originally written as such and this allowed it to be utilized for multiple asteroids without the need to modify and recompile each time. However, there were several other modifications that were needed in order for *volInt.c* to be compiled and executed properly.

The first main modification dealt with the outdated exit calls within the program. When the *volInt.c* code was written in 1996 it was able to utilize the implicit declaration of the function exit by using declaring `#include<stdio.h>` as one of the main packages. However, modern compilers show that this is not the case anymore. Running the included compiling command `cc volInt.c -O2 -lm -o volInt` will now produce an error on modern machines as follows:

```
volInt.c:129:5: warning: implicit
declaration of function 'exit'
[-Wimplicit-function-declaration]
 129 |     exit(1);
      |     ~~~~
+++ |+#include <stdlib.h>
 47 |
```

This occurrence happens on lines 129 & 326, yet as can be seen from the output of the compiler this exit function is now located within the `<stdlib.h>` package. Thus, the addition of the line `#include <stdlib.h>` allowed for the program to use the implicit declaration of the exit function on modern machines.

This was not the only main error that came from the compiler, as the next error message was produced during the initial compiling of *volInt.c* was as follows:

```
volInt.c:235:2: warning: return type
defaults to 'int' [-Wimplicit-int]
 235 |   compFaceIntegrals(FACE *f)
      |   ~~~~~~~~~~~~~~~~~~~~~~
```

This was due to the fact there was not a return value from the `compFaceIntegrals`

function within the code and is simply used for other calculations. In order to resolve this issue the `void` command needed to be added to the line as `void compFaceIntegrals(FACE *f)` which tells the program to disregard the error shown above [7]. With these main modifications, the compiler was successfully able to create the *volInt.exe* program.

The last modification that was needed did not come from compiling errors, but rather from the initial declaration of limits on the maximum number of vertices and faces. The lines `#define MAX_VERTS 100` & `#define MAX_FACES 100` needed to be adjusted in order to allow for more complex polyhedron models to be used.

2.3 Analyzing the *volInt.exe* output

As stated previously the main assumption used within this project is that each output of *volInt.exe* was given with a density of 1.0, yet this did not affect the volume and CM calculations as they were independent of the density. However, each inertia tensor calculated for the asteroids was dependent on density. This means that each inertia tensor carried the density $\rho = 1.0$ with assumed units. With this in mind, the following is the output created by *volInt.exe* for Apophis (99942):

```
T1 =                +1.274109
Tx =                -0.002739
Ty =                -0.001364
-----etc-----
Tyz =                -0.000151
Tzx =                -0.000918
center of mass:
(  -0.002150,   -0.001070,   -0.000308)
inertia tensor with origin at c.o.m. :
+0.179997        -0.000572        +0.000919
-0.000572        +0.290829        +0.000151
+0.000919        +0.000151        +0.318391
```

As can be seen the CM is not centered to zero for an irregular body as expected. Furthermore, it should be noted that the volume calculation $T1 = +1.274109$ is also in assumed units of the given OBJ file. If the approximate density is known, including units, it can be added to the *volInt.c* code which will in turn give the volume proper units. Yet, there is still a problem with assumed units within the shape models. This brought about the need for dimensional analysis when testing the software to determine the proper scaling of the asteroid shape models.

3 Plotting Asteroids as Tetrahedron CM

Dr. Diogo Sanchez’s previous work on Apophis (99942) showed that the gravitational potential for the asteroid could be calculated as a summation of each individual tetrahedron’s CM within MASCON I [8]. This served as the main guide to ensure that the Asteroid models presented within the GUI accurately represented the asteroid’s irregular bodies. Furthermore, it served as a properly scaled model of Apophis (99942) as compared to initial calculations using OBJ files with assumed units of distance for the vertices.

3.1 Calculating Each Tetrahedron’s CM

In order to begin using the MASCON method of analysis the CM for each tetrahedron must be calculated which can be used for calculating gravitational fields and plotting orbits around the desired asteroid for analysis. For this project, the asteroid shape models are presented as the collection of these CM points. This was determined analytically as follows [9]:

$$CM = \left(\frac{(x_1 + x_2 + x_3 + CM_{p,x})}{4}, \right. \quad (1) \\ \left. \frac{(y_1 + y_2 + y_3 + CM_{p,y})}{4}, \right. \\ \left. \frac{(z_1 + z_2 + z_3 + CM_{p,z})}{4} \right)$$

where the points $CM_{p,x}$, $CM_{p,y}$, & $CM_{p,z}$ are the points of the polyhedron’s CM. This is because the asteroid shape model center is the 4 vertex of each tetrahedron with the other 3 vertices connected to the surface’s face.

The next step was to code this into Python to calculate each CM and write it to a data file for the GUI. Initially, this was achieved with the program *Apophis_Recreation.py* that would be later adapted into *Asteroid_CM.py*. As Apophis (99942) was used as a benchmark for the project the code reflected this, yet with minor modifications it was adapted to be used for any asteroid shape model given the input of the polyhedron’s CM calculated for *volInt.exe* and the asteroid’s OBJ file.

The code for *Asteroid_CM.py* uses Eq. 1 for finding the individual CMs for each tetrahedron. The first step in utilizing this equation was to understand that each of the 3 vertices that are being used in the calculations, alongside the polyhedron’s CM, are designated to each specific tetrahedron. This designation is reflected within the face data, an example of which can be seen in the Apophis (99942) OBJ file. The first tetrahedron is made using the vertices 1, 2, and 3; while the second tetrahedron uses the vertices 1, 3, and 4. This continues for each of the 2024 faces or individual tetrahedrons that form the polyhedron shape model. In order for this to be reflected in calculations each vertices must be indexed using the faces. To achieve this a for loop was created using the faces as the indices and the total length or number of faces as the range of calculations. The following portion of code is an example of this calculation from

Asteroid_CM.py; note that the line continuation character ‘\’ was added for spacing:

```
data_size=np.shape(Face_Data)[0]
#
for ii in range(0,data_size):
    Center_mass_calc_x = \
        (Vert_Data[Face_Data[ii,0],0]\
        + Vert_Data[Face_Data[ii,1],0]\
        + Vert_Data[Face_Data[ii,2],0]\
        + CM[0])/4
```

This was repeated for the y and z-coordinates to obtain all CM coordinates within the 3D space. Notice that this produces a total of 2024 CM points, as shown in Fig. 2, for each of the tetrahedrons within the Apophis (99942) shape model as designated by the face data, and utilizes the CM as the 4th point as the center of the polyhedron. With this calculation, the MASCON I model was created, yet the next step was to plot the Apophis shape model to determine the calculation’s accuracy.

3.2 Plotting Apophis

Once the calculations of the CM were performed using the Apophis (99942) OBJ file, the calculated CM for the polyhedron shape model was plotted and compared to the previous model used within Dr. Diogo Sanchez’s work. This model is shown in Fig. 1 and was scaled at $\gamma = 0.285 \pm 0.158$ to maintain the proper diameter of 0.387 km [8]. This scaling was then used to create the model in Fig. 2 which is identical to the reference model shown in Fig. 1. This showed that the CM calculations from Eq. 1, and implemented within *Apophis_Recreation.py*, were correct as they yielded the desired result. However, as the development of the universal *Asteroid_CM.py* program from *Apophis_Recreation.py* began considerations had to be made for a simplistic

method of scaling for each of the subsequent models that would be included within the final project.

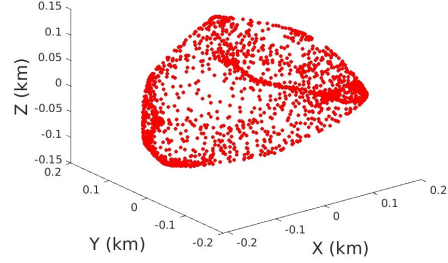


Fig. 1: Apophis (99942) model from “*First Approximation for Spacecraft Motion Relative to (99942) Apophis*” [8]

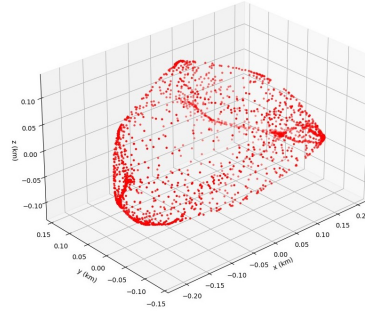


Fig. 2: Apophis (99942) model from “*Apophis (99942)*” created within *Apophis_Recreation.py* using the scaling of $\gamma = 0.285$

3.3 Creating a Universal Method of Scaling

Each model within the 3d-asteroids.space catalog not only carries assumed distance units but also a disclaimer from the author stating that “The author reserves the right not to be responsible for the topicality, correctness, completeness or quality of the information provided” [1]. This is due to the fact that not all shape models were modified to their respective asteroid’s scale when created. As shown in Fig. 3 this is the raw model of

Apophis (99942) taken directly from the OBJ file and is approximately 4 times larger than it should be.

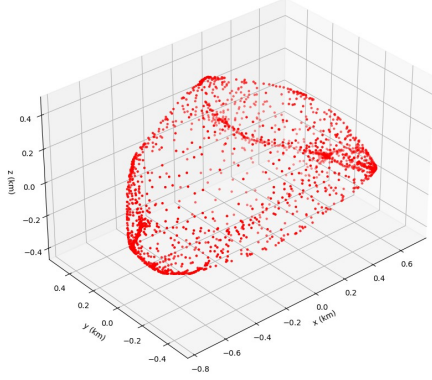


Fig. 3: Apophis (99942) shape model without scaling

Even though the correct scaling for Apophis (99942) was determined to be $\gamma = 0.285 \pm 0.158$ this does not apply to every other asteroid shape model. In order for *Asteroid-CM.py* to be truly universal a method for scaling was determined based on ratios of mean radius for the accepted value and calculated value from the OBJ file as shown below:

$$Scale = \frac{Accepted\ Mean\ Radius}{Calculated\ Mean\ Radius} \quad (2)$$

This was then added into the program and allowed for the CM output data file to be scaled accordingly. For Apophis (99942) the approximate diameter of $0.387\ km$ was used [8]. This value along with Eq.2 yielded a mean radius value of $\gamma \approx 0.2388$ from the OBJ file and was used to create Fig. 4 which serves as the correct model for this project. This is inaccurate from the correct scaling value by approximately 16.21% yet still falls within the standard error of ± 0.185 and yields a significantly more accurate model than the one shown in Fig. 3 which is why the model presented in Fig. 4 was used for the final GUI.

In order to implement this in the code a user prompt and (Y/N) selection was added

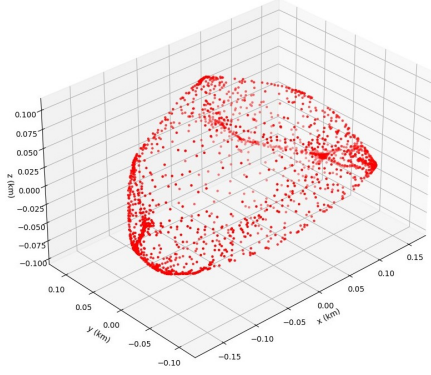


Fig. 4: Apophis (99942) represented as the 2024 Center of Mass points for each tetrahedron with scale $\gamma \approx 0.2388$

to check if the user wanted to use the scaling function. Then an `if & elif` statement is used to either calculate the scaling and multiply the output data to match the scaling, or the data is left un-scaled and the user is warned with a disclaimer that the output is not a scaled representation of the asteroid. The following is an excerpt of the calculations inside the `if` statement:

```
vec_x = Vert_Data[:,0] - CM[0]
vec_y = Vert_Data[:,1] - CM[1]
vec_z = Vert_Data[:,2] - CM[2]
Radius_Calc = np.sqrt(vec_x**2 \
                      + vec_y**2 + vec_z**2)
# Calculate mean radius from OBJ file
Mean_radius_calc = np.mean(Radius_Calc)
# Calculate the proper scale
Scale = Radius_input/Mean_radius_calc
-----
# Scale Outputs
Vert_Data_mesh = Vert_Data*Scale
Output_Array_CM = np.array(Center_mass_tetra)\
                    *Scale
```

As can be seen the calculations use a `Radius_input`, which is taken from user inputs, and relates this as a ratio to the calculated mean radius `Mean_radius_calc`. This is then used to scale the array that will be written to the `.out` file for the tetrahedron's

CM locations. This scaling was then included within the GUI in order to allow the face mesh to also be scaled properly.

3.4 Adding Face Mesh to Plot

To give the shape models more perspective the face mesh that defines the outside of the shape was added using `Poly3DCollection()` from the `mpl_toolkits.mplot3d.art3d` within matplotlib [10]. This function uses the vertices as inputs, and similar to the CM calculations the line of code uses the face data as each vertices respective positions as follows:

```
Poly3DCollection(
[Vert_Data[iii] for ii in Face_Data])
```

Additional parameters included in `Poly3DCollection()` help define the edge colors along with the parameters such as line width. This was included in *GUI_packages.py* which defines a function `OBJ_READ_IN()` that returns `Asteroid_Mesh` based on the vertex and face data from the OBJ file. In addition, the function also takes a `scale` variable which if set to 1 does not scale the model. The function was imported as `GP` and called into the main GUI within each plot definition to create the face mesh that defines the asteroids outside and shows this along with the internal CM positions. An example of this being implemented is as follows:

```
scale = 0.22834907252107323
Asteroid_Mesh = GP.OBJ_READ_IN(
    'Apophis.obj',
    scale)
```

Here we see that the calculated scaling of Apophis taken from *Asteroid_CM.py* was included in order to scale the face mesh to the CM particle cloud thus producing a complete shape model for the asteroid.

4 Creating Friendly User-Friendly GUI

For this project, PyQt5 and Pyside6 were used to create the GUI as it was able to produce the latest Windows 10 interface. In order to understand how to embed 3D plots using matplotlib an example from the Qt for Python documentation was adapted and used to create the GUI [11]. In order to plot each asteroid the `QComboBox()` function was utilized for a drop-down menu for plot selection. Each selection within this menu will then call the individual plots made from the `.out` files produced by *Asteroid_CM.py* along with their OBJ files which form the cloud of particles representing each tetrahedron's CM and the outside mesh of faces for the shape model respectively.

4.1 User Friendly GUI Design

To make the plot interface easy to use, azimuth and elevation controls were used from the GUI example for fine adjustments of plot adjustments. These can be used by the user along with built-in mouse actions for plot rotation and zoom. For more controls, the matplotlib toolbar was included using the following lines:

```
# Embed Matplotlib toolbar
self.toolbar = NavBar(self.canvas,
                        self)
-----
llayout = QVBoxLayout()
llayout.addWidget(self.toolbar)
```

This defines the the toolbar into `self` which is the GUI class object, and the latter portion places this toolbar on top of the plot as it is declared prior to the rest of the left layout or `llayout`.

4.2 Accommodations for the Colorblind

As this is educational software made accessible to the public, considerations were made to the plotting colors. This is because there are many colorblind citizens in the public, and those with this condition have difficulties when discerning specific colors. That is why an accessible color palette was used to select a pair of contrasting colors that anyone could distinguish from one another.

In order to design the color scheme Dr. David Nichols' *online colorblindness simulation tool* was utilized to choose a color palette that could be discernible for anyone with the 3 main types of colorblindness protanopia, deuteranopia, or tritanopia [12]. Using the chosen color palette shown in Fig. 5 each plot was tested with the simulated colors which can be seen in Fig. 6 to prove that each plot would be visible no matter which colorblindness someone would have.

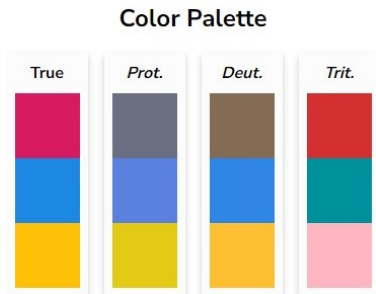


Fig. 5: Chosen color palette

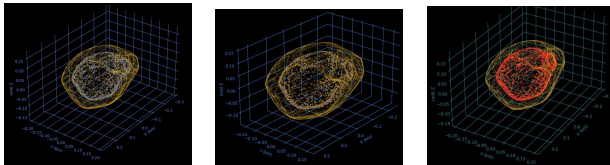


Fig. 6: Plots with simulated colorblindness from left to right: protanopia, deuteranopia, or tritanopia

4.3 Glitches in the GUI

A glitch was found within the GUI during the testing phase which caused traces of the previous plot's axes to remain within the figure margins. This was a key error to remedy as this left some of the axes illegible. An example of this error can be seen in Fig. 7 where the axes overlap from the previously selected plots, and in addition causes the program itself to lag and freeze when trying to use any of the plot controls for rotation, zoom, or any other movement.

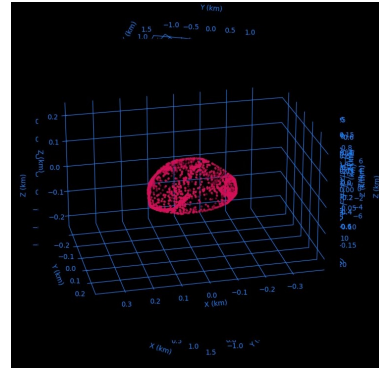


Fig. 7: An example of the figure glitch within the GUI

The main fix that needed to be implemented was the use of the figure clearing function `fig.clf()` within the canvas configuration definition. This needed to be called before setting up the next figure to ensure no traces of the previous figure's axis would show in the next selected plot.

4.4 Adding Educational Descriptions for Asteroids

The last addition to the main program was to incorporate descriptions of each asteroid along with its shape model. These descriptions were cited within the program itself, and teach the user about several features and history of the asteroids. Each description was

written in Markdown as it creates an organized text output for the user including headers and other style choices.

To add the descriptions in the `QTextEdit()` function was set as `Asteroid_info` and added to the right layout below the asteroid selection dropdown menu. The `Asteroid_info` was then set to each individual file within each plot definition so that selecting a new asteroid would tell the program to call the new Markdown file along with the plot. This was done by using the `with open()`, `.read()`, and `.setMarkdown()` functions so assign the files to `Asteroid_info`.

5 Conclusion

The finished project contains 26 total asteroid shape models that were downloaded from the 3d-asteroids.space catalog and processed into MASCON I using `volInt.c`, `obj2MirtichData.py`, and `Asteroid.CM.py`. These MASCON I models were incorporated into `GUI_Capstone.py` which serves as the GUI's main program. The `.out` files produced were included along with the OBJ files and Markdown descriptions files to produce a cohesive shape model and educational description inside the GUI. Additionally, the `GUI_packages.py` was created to handle additional function definitions for reading in these files and included scaling of the face mesh to match the CM scaling.

References

- [1] Greg Frieger, "3d asteroid catalogue," 2013-2020 [Online].
- [2] "Alias/wavefront object (.obj) file format," 2023 [Online].
- [3] T. Chanut, S. Aljibaae, and V. Caruba, "Mascon gravitation model using a shaped polyhedral source," *MNRAS*, vol. 450, pp. 3742–3749, July 2015.
- [4] J. Pearl and D. Hitt, "Mascon distribution techniques for asteroids and comets," *Celest. Mech. Dyn. Astron.*, vol. 134, Dec. 2022.
- [5] B. Mirtich, "Fast and accurate computation of polyhedral mass properties,," *J. graphics tools*, vol. 1, pp. 31–50, Jan. 1996.
- [6] "Openfoam-2.1.x," 2023 [Online].
- [7] "What is void in c," 2023 [Online].
- [8] S. Aljibaae, D. M. Sanchez, A. F. B. A. Prado, J. Souchey, M. O. Terra, R. B. Negri, and L. O. Marchi, "First approximation for spacecraft motion relative to (99942) apophis," *Romanian Astron. J.*, vol. 31, pp. 241–263, Nov. 2021.
- [9] J. Burkardt, "Computational geometry lab: Tetrahedrons." Information Technology Department Virginia Tech, Dec. 2010.
- [10] John Hunter and Darren Dale and Eric Firing and Michael Droettboom and the Matplotlib development team, "mpl_toolkits.mplot3d.art3d.poly3dcollection," 2023 [Online].
- [11] "Matplotlib widget 3d example," 2023 [Online].
- [12] David Nichols, "Coloring for colorblindness," 2023 [Online].

Appendices

A Software

The following website contains the download for installation of the software:

🔗 https://github.com/evan-a-blosser-1/Asteroid_Shape_Model_GUI_1.0/releases/tag/Version_1.0

All programs used for this project will be made open source and are available at the following Github repository:

🔗 https://github.com/evan-a-blosser-1/Asteroid_Shape_Model_GUI_1.0

The software located in this repository is as follows:

- *volInt.c* includes untouched zip file, and modified file
- *obj2MirtichData.py*
- *Asteroid_CM.py*
- *GUI_packages.py*
- *GUI_Capstone.py*

The data files used for the GUI and located in the repository in the following folders:

- Descriptions
- OBJ shape models
- OUT Data Files

The Software icon is designated by the following pixel art (special thanks to Clayton G. Blosser for providing the art)



Special thanks to Dr. Diogo Merguizo Sanchez for being my mentor during this project.

B Engineering Physics Design Process: Addressing the 12 criteria

B.1 Define a Problem

The main problem we face is insufficient knowledge about analyzing asteroid gravitational fields with the lack of exact shape models of their irregular bodies. This brought about the need to use MASCON to create a precise method of analyzing gravitational fields with irregular bodies.

B.2 Brainstorm

The starter problem that developed during meeting brainstorm led to the development of a simple educational software written in Python that contained shape models in MASCON I. Ideas were presented within meeting presentations and logged into a research notebook. This was a good first step to the main solution as these models could be used for gravitational field analysis and developing models into MASCON III & VIII.

B.3 Research

Research began with the *volInt.c* program and how to properly apply it to asteroid shape models. However, this program does not give any other analysis of individual tetrahedrons CM. This led to researching how to create a universal program that could process the models into MASCON I. Research then led to how to present this as educational software in a user-friendly interface.

B.4 Identify Criteria and Specify Constraints

A main constraint with developing the program was how it could be beneficial within the short time frame. For future research, this will lead to analyzing gravitational fields using the models created yet this would take more years of research. Thus, the constraint to create an educational program that would teach those at the high school and college level about asteroids was put into place. The solution became creating asteroid models in MASCON I and presenting them in a user-friendly interface, along with descriptions of each asteroid that would teach others about the wonders of space.

B.5 Explore Possibilities

The development of an educational program was a very beneficial tradeoff, as this allowed for the MASCON I models to be utilized while also developing a substantial end product. The possibility of creating a user-friendly interface was explored and found to be an ideal solution and first step to the overall problem of analyzing gravitational fields. Exploration of modifying the *volInt.c* program to output individual tetrahedrons CM in MASCON was also considered as this would allow for ease of processing asteroid shape models.

B.6 Select an Approach

The main approach selected was to create software that could process models as the modification of *volInt.c* to the degree needed for developing MASCON was found to be out of the scope of this undergraduate research. Thus, developing *Asteroid_CM.py* and using the shape models created by the program in an educational manner was found to be the better choice.

B.7 Develop a Design Proposal

The development began with creating the program *Apophis_Recreation.py* which would later be developed into a universal program *Asteroid_CM.py*. The design proposal for the user-friendly interface also began with an outline of what information the software would display to the user. Furthermore, the choice of using PyQt5 and Pyside6 was made to create a user-friendly interface and present an educational program.

B.8 Make a Model or Prototype

The prototype was *Apophis_Recreation.py* as this served as a benchmark for the software. The model created could be compared to previously made models and show that a universal program could be created to process any asteroid model.

B.9 Test/Evaluate Design

Testing was performed on *Apophis_Recreation.py* and several iterations were made after the evaluation of the model showed it to be inaccurate. Once the shape was found to be accurate the scaling was analyzed and a method of universally scaling any shape model file was designed.

B.10 Refine the Design

The refined program was *Asteroid_CM.py* where 26 total models were processed without issue. Documentation included comments within the code as well as a **README** file. The GUI was also refined in order to present educational content, and considerations for colorblindness were included for the shape models presented to allow for accessibility for those with the ailment.

B.11 Create or Make Solution

The final program was then packaged with a **.spec** file that could be installed using PyInstaller. This allowed for mass distribution of the software to the public. A GitHub was also created to allow for the code to be open source and accessible to the public.

B.12 Communicate Processes and Results

This paper serves as a detailed process of how the software was developed, and a 25-minute presentation was conducted as a final talk over all the details of this project. For the educational program all that is left unfinished is creating a `onefile` that could be placed on the desktop outside of the `dist` folder created by PyInstaller.

C The Eight Considerations

C.1 Economic

This is an open-source software created without any monetary constraints. The economics of this was non-existent as the code was created in free-to-use Interactive Development Environments (IDEs), and was made accessible to the public free of charge under an MIT license.

C.2 Environmental

This program is very environmentally friendly as its production was made with pre-existing computers. Besides the small amount of electricity needed to power the computer no other resources were needed.

C.3 Sustainability

This is a fully open-source project that can be accessed by anyone. Thus, this will hopefully have many years of sustainable educational benefit.

C.4 Manufacturability

As this is software the manufacturing is already complete.

C.5 Ethical

This is ethical as all information used for the research and development of the software has been cited. Furthermore, it is made accessible to anyone in the world with a computer.

C.6 Health and Safety

Health and safety were not any major issues as this was developed in a computer lab.

C.7 Social

This software will have a positive impact on society as it can educate people about asteroids.

C.8 Political

As this is a fully developed software, there are not any political considerations to be made.